

Patriot Pot: High-Fidelity Threat Intelligence via Cloud-Native Deception Systems

A Longitudinal Analysis of Adversary Behavior in Hardened SSH Environments

Kevin Landry — Group Lead & Cloud Security Systems Architect

Roshan Raj — Security Analyst

Abdulhamid Alhumaid — Systems Visualization

College of Engineering and Computing
George Mason University
Fairfax, VA, USA

Abstract

As cloud computing infrastructures cement themselves as the primary backbone of the modern digital enterprise, they have simultaneously become focal points for automated adversarial reconnaissance. Traditional perimeter defenses (firewalls, IDS) provide binary block/allow logic but fail to capture the intent and Tactics, Techniques, and Procedures (TTPs) of modern adversaries. This project, titled “Patriot Pot,” details the deployment, hardening, and analysis of a cloud-native deception system using a modified Cowrie honeypot within an Amazon Web Services (AWS) Virtual Private Cloud (VPC). Over a 48-day monitoring period, the system captured 109,024 discrete events and identified 2,338 unique attacker IP addresses. By systematically hardening the honeypot’s artifacts—specifically altering default SSH banners and filesystem fingerprints—the research team increased attacker dwell time, achieving a 59.5% login success rate. This high interaction rate allowed capture of zero-day behavioral patterns, including the correlation of specific

“bait” files (e.g., rat_loader_v5.py) with subsequent payload execution. The findings argue that customized, medium-interaction honeypots provide superior threat intelligence compared to static logging, offering a viable path toward proactive cloud defense.

Index Terms

Cloud Security, Honeypot, Cowrie, Threat Intelligence, AWS, Deception Technology, IoC, TTPs, MITRE ATT&CK, NIST CSF, Active Defense.

I. Introduction

A. The Evolving Threat Landscape

The cybersecurity landscape of 2025 has evolved into a complex domain where traditional defensive mechanisms are increasingly insufficient. The rapid migration of enterprise workloads to cloud infrastructures has fundamentally altered the defensive perimeter. No longer bounded by on-premise firewalls, the perimeter has dissolved into a nebulous collection of APIs, microservices, and virtual instances. Major Cloud Service Providers (CSPs) like AWS operate under a “Shared Responsibility Model,” securing the underlying cloud infrastructure while customers remain responsible for securing their own operating systems, applications, and data. This gap has led to a proliferation of misconfigured Secure Shell (SSH) services and exposed management ports—low-hanging fruit aggressively targeted by global botnets.

As noted by Morić et al. in their analysis of advancing cybersecurity strategies, modern threats necessitate preemptive actions to safeguard digital assets [1]. Firewalls and intrusion detection systems (IDS) still operate on binary logic; they allow or block traffic based on signatures or access control lists. While essential for baseline security, these tools fail to capture the intent or adaptability of an adversary. When a firewall blocks a connection, the defender learns nothing about what the attacker would have done if allowed through. The intelligence regarding which tools would have been downloaded, which kernels exploited, or which commands executed is irretrievably lost.

B. The Paradigm of Active Defense

To bridge this intelligence gap, deception technology—specifically honeypots—has emerged as a critical component of active defense strategies. Active defense flips the script by engaging the adversary rather than passively deflecting them. By deploying a realistic decoy environment (a “Patriot Pot”), defenders can observe attacker behavior in a controlled setting, catalog the tools and malware used, attribute attacks via threat feed correlation, and ultimately deter future attacks by increasing the cost and complexity for adversaries. This operationalizes the concept of Active Defense within a cloud-native context. Unlike static perimeter controls, a honeypot invites malicious interaction, turning the attacker into an unwitting source of intelligence for the defender.

This paper presents the findings of the Patriot Pot project, a research initiative designed to evaluate the efficacy of a hardened Cowrie SSH honeypot in a live AWS environment. The contributions of this work are threefold: (1) a detailed architecture for deploying secure, isolated honeypots in AWS, (2) an analysis of 109,000+ events, classifying adversaries into distinct behavioral clusters, and (3) evidence supporting the hypothesis that artifact customization increases attacker engagement and yields higher-fidelity intelligence.

II. Background and Related Work

A. Deception Theory in Cybersecurity

The concept of deliberately baiting an adversary predates the digital age. In law enforcement, sting operations place enticing opportunities to lure criminals into revealing themselves—a tactic analogous to cyber honeypots. In computing, an early formalization appeared as far back as 1976: Federal Information Processing Standard 39 defined entrapment as “the deliberate planting of apparent flaws in a system for the purpose of detecting attempted penetrations or confusing an intruder” [3]. This perfectly encapsulates the modern honeypot: a system with no production value, designed solely to be attacked. Any interaction with a honeypot is, by definition, unauthorized and malicious. Lance Spitzner’s seminal work on honeypots further emphasized that their value lies strictly in the interaction they capture, since any traffic to a honeypot is inherently suspect [2].

Deception in cybersecurity is fundamentally about increasing the cost of an attack while decreasing the cost of defense. By deploying false targets and misinformation, defenders aim to disrupt the adversary’s decision cycle. Honeypots exemplify this: they waste attacker time and resources and can even feed attackers spurious data. The success of deception relies on believability—the honeypot must appear enough like a legitimate system to entice deeper engagement from the adversary.

B. Evolution of Honeypot Technology

Honeypots are generally categorized by interaction level. Low-interaction honeypots simulate only limited aspects (e.g., network services) and are easily scalable but provide minimal data (and are easily identified). High-interaction honeypots involve real operating systems and full services, yielding rich data at the expense of high risk (an attacker may compromise the system) and maintenance overhead. Medium-interaction honeypots strike a balance by emulating a system’s functionality without running a full OS; they offer more fidelity than low-interaction sensors while limiting risk.

Cowrie, the platform used in this project, is a medium-interaction SSH/Telnet honeypot. It emulates a Unix-like shell environment in Python, recording brute-force login attempts and shell interactions without exposing a real kernel to the attacker. Medium-interaction designs like Cowrie allow attackers to execute commands in a sandboxed filesystem, capturing their actions in detail while preventing actual harm to the host system.

Early implementations demonstrated honeypot value. In *The Cuckoo’s Egg* (1989), Clifford Stoll famously set up fake government files to ensnare a KGB-affiliated hacker long enough to trace them [4]. In the early 1990s, Bill Cheswick described a high-interaction honeypot where he isolated an attacker in a Unix system and monitored their keystrokes live. These pioneering efforts underscored how much insight can be gained by “letting an intruder in.” Modern honeypots build on these principles with more automation and realism, integrated with threat intelligence systems for faster analysis.

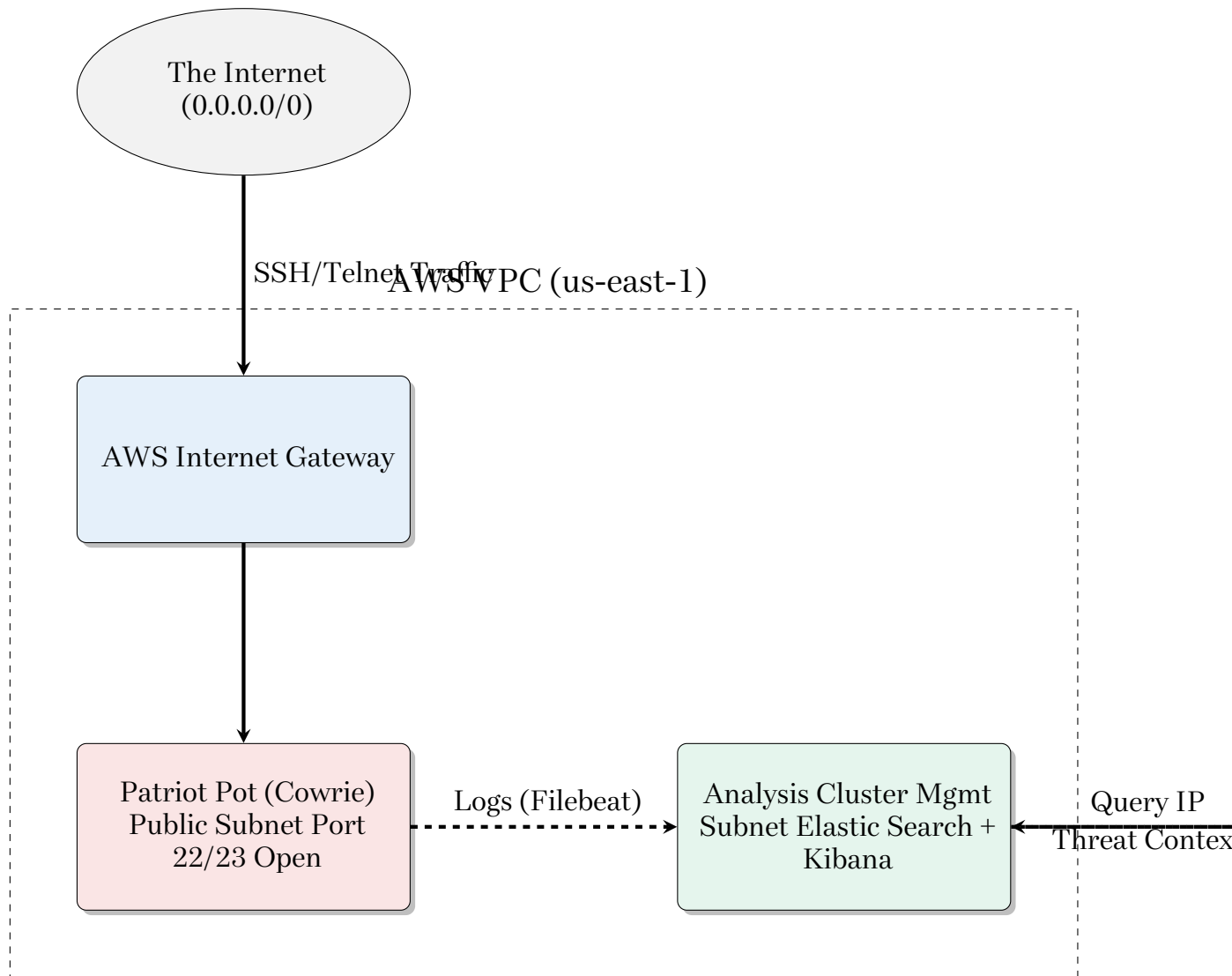


Fig. 1: High-Level Architecture of Patriot Pot. The sensor (Cowrie) sits in a public subnet, forwarding telemetry to a secured management subnet for enrichment and analysis.

C. The Fingerprinting Challenge

A critical challenge for honeypots is avoiding detection by attackers. If a honeypot’s presence is obvious, skilled adversaries will simply disconnect, depriving defenders of data. Unfortunately, many honeypots (especially out-of-the-box configurations) exhibit telltale signs. Services like Shodan run continuous Internet scans and even provide a “Honeyscore” rating – a probability that an IP is a honeypot – based on such fingerprints.

Prior research has shown that default Cowrie installations are highly susceptible to fingerprinting. Krajčík et al. (2025) note that static filesystem structures, default login banners, and identical command responses across Cowrie instances make them easy to identify [6]. For example, Cowrie’s default SSH banner (SSH-2.0-Twisted) is a dead giveaway – it references Twisted (the underlying Python framework) rather than a real OpenSSH service. Similarly, the default fake filesystem often contains directories or files that would not exist on a real production server. Attack tools and bots increasingly incorporate checks for these indicators. When an automated scanner detects such signatures, it may terminate the session to avoid tipping off defenders, or even feed false data to the honeypot (honeypot poisoning).

D. Cloud Deployment Context

Deploying honeypots in cloud environments introduces unique opportunities and concerns. Cloud platforms enable rapid provisioning of isolated test environments, but also come with strict responsibility delineation. Under AWS’s Shared Responsibility Model, AWS secures the cloud infrastructure, while the customer must secure what’s in the cloud (OS, applications, data). Our honeypot takes advantage of this by intentionally simulating a negligent customer deployment – e.g., an EC2 instance with default credentials and open ports – while enclosing it in a secure VPC sandbox. This attracts opportunistic attackers (appearing to them as an easy target) but minimizes risk to any production assets.

III. Theoretical Framework: A Game of Hide-and-Seek

The interplay between a honeypot and an attacker can be modeled as a game of incomplete information, where each side tries to maximize its payoff while guessing the other’s strategy. In our context, it is a zero-sum game: time and information that the defender gains are time and information lost to the attacker. The attacker’s goal is to compromise as many hosts as possible while minimizing resource cost and avoiding detection. The defender’s goal (via the honeypot) is to maximize intelligence gathered (attacker dwell time and data) while minimizing risk to real systems.

This can be viewed through a game-theoretic lens: If the honeypot is too obviously fake or too easy (low-interaction, trivially fingerprintable), a rational attacker will minimize cost by disconnecting immediately. If the honeypot is too secure or too difficult (e.g., a fully patched system), the attacker may also give up quickly, seeking softer targets. The equilibrium—our target—is a system that appears vulnerable enough to be worth exploiting, yet realistic enough to sustain the adversary’s interest.

Hypothesis: By modifying specific artifacts (SSH banners, filesystem content, hostnames, certificates, etc.), we can shift this equilibrium in favor of the defender. If done correctly, attackers will invest more time and reveal more of their TTPs before realizing they are in a decoy environment.

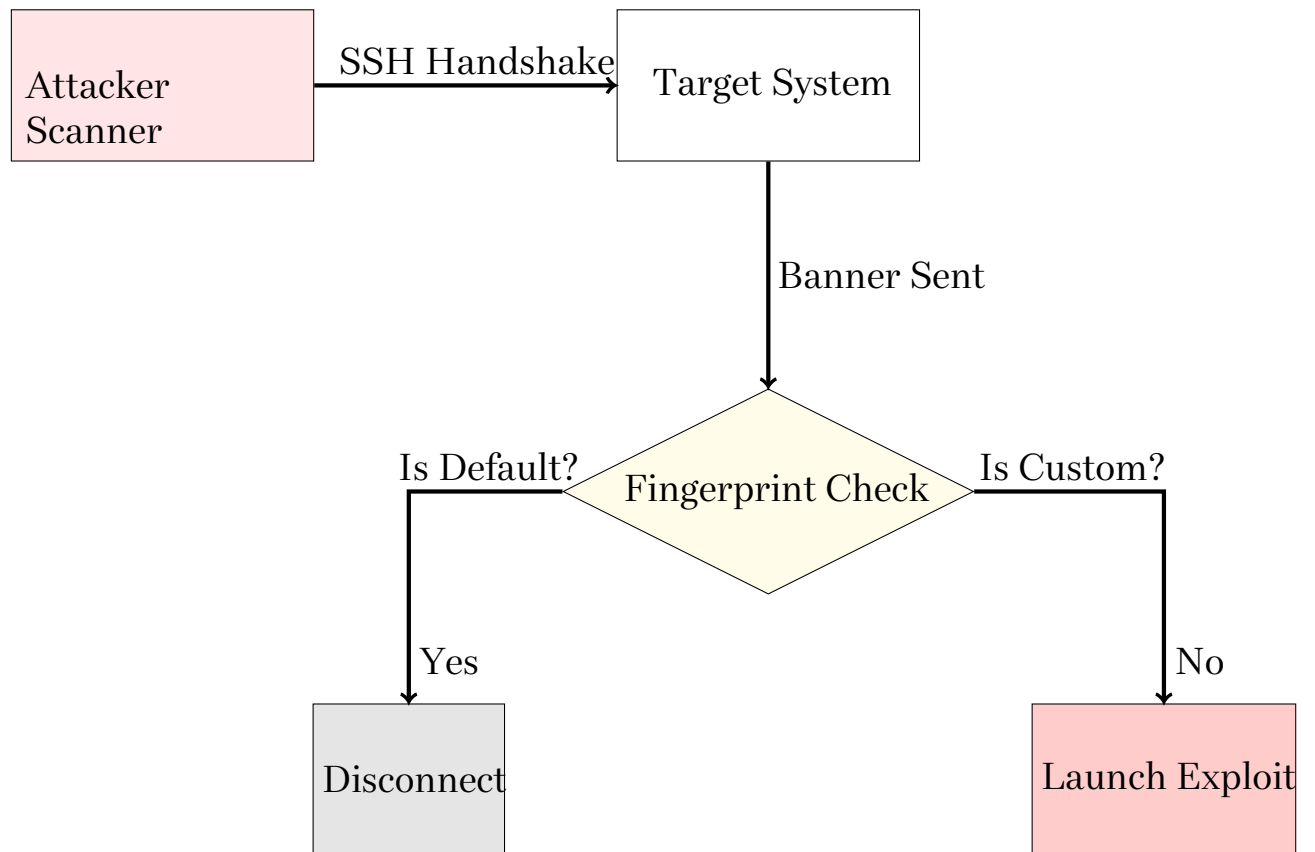


Fig. 2: The Fingerprinting Decision Loop. Default honeypots cause scanners to disconnect immediately, resulting in lost intelligence. Hardened artifacts force the attacker to proceed.

IV. Methodology and System Architecture

This section details the technical implementation of the Patriot Pot system. The architecture was designed to meet two conflicting requirements: accessibility (to attract attackers widely) versus isolation (to protect the hosting environment and prevent any attacker pivot).

A. Cloud Infrastructure Design (AWS)

The environment was hosted in AWS’s us-east-1 region, chosen for its high density of cloud assets. We provisioned a t3.medium EC2 instance running Ubuntu 22.04 LTS as the honeypot server.

- 1) **Network Topology & Segmentation:** To prevent honeypot breakout, we implemented strict network isolation following a Zero Trust model. The VPC contained two subnets: a Public Subnet for the honeypot and a Management Subnet for analysis components. An Internet Gateway (IGW) attached to the VPC allowed external traffic only to the public subnet.
- 2) **Security Group Configuration:** We used AWS Security Groups to enforce ingress and egress policy.
 - **Ingress:** GMU-Honeypot-SG allowed inbound TCP port 22 and 23 from 0.0.0.0/0.
 - **Egress:** The honeypot instance was only permitted to make DNS queries (UDP/53) and HTTP/HTTPS requests (TCP/80, 443). SMTP (port 25) and all other common outbound vectors were blocked to prevent liability.

B. The Deception Sensor – Cowrie Implementation

Cowrie was selected as our honeypot platform. We executed a comprehensive hardening strategy using a custom deployment script (deploy-fix-remote.sh).

- 1) **Artifact Masquerading:** Table I summarizes key artifacts we modified to avoid detection.

TABLE I: Cowrie Honeypot Artifact Hardening Matrix

Artifact	Default Behavior	Hardened Configuration
SSH Banner	SSH-2.0-Twisted	SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.5
Filesystem	Minimal fake FS	Populated /home/admin, /home/support with realistic files
Hostname	srv01 or cowrie	ip-172-31-42-19 (AWS convention)
Bait Files	None	/tmp/rat_loader_v5.py, /var/backups/db_dump.sql
SSL Cert	Self-signed “Cowrie”	Custom OpenSSL certificate

C. Intelligence Pipeline

Raw honeypot logs are voluminous and noisy. To convert this data into actionable intelligence, we implemented a multi-stage pipeline comprising log aggregation, enrichment, and real-time alerting.

- Ingestion (Elastic Stack): Filebeat shipped JSON logs to a centralized Elastic Cloud instance.
- Enrichment (Python Middleware): A custom script (`enhanced_monitor.py`) enriched IPs using GreyNoise (to filter benign scanners) and Shodan (for geolocation).
- Alerting (Discord): Critical events triggered real-time webhooks.

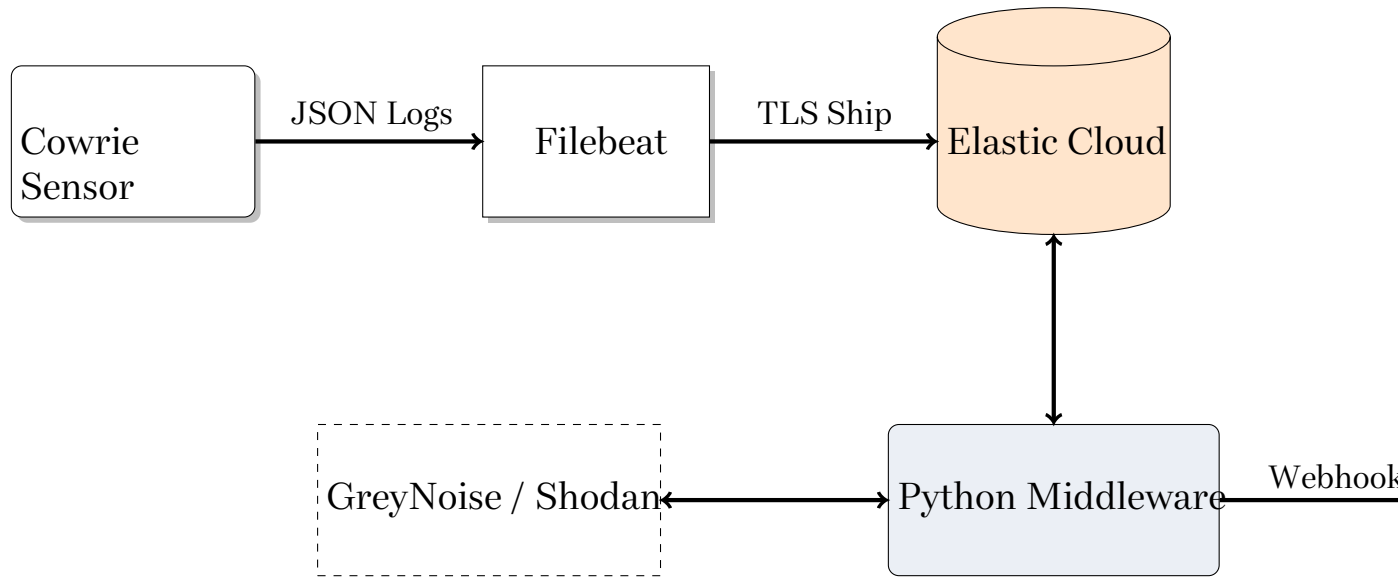


Fig. 3: The Intelligence Pipeline. Data flows from the sensor to Elastic, where it is queried by middleware for enrichment and alerting.

V. Experimental Results and Forensic Analysis

A. Quantitative Overview

Over the 48-day operational window, Patriot Pot captured 109,024 discrete log events.

B. Geographic and Temporal Distribution

Attacks originated from 42 distinct countries. The top 5 were China (~31%), Russia (22%), the United States (18%), Brazil (7%), and the Netherlands (5%).

TABLE II: Operational Statistics Summary (48 Days)

Metric	Count	Description
Total Events	109,024	Discrete log entries
Connection Attempts	16,114	TCP connections (Port 22/23)
Unique Attacker IPs	2,338	Distinct source IP addresses
Successful Logins	9,599	Compromises via weak creds
Commands Executed	9,409	Post-login shell interactions
Login Success Rate	59.5%	Result of weak credential policy

C. Attacker Behavior Kill Chain Reconstruction

By analyzing the sequence of event types in each session, we reconstructed a generalized attack kill chain within the honeypot.

- Reconnaissance (15%): Initial TCP connection handshakes and banner grabbing.
- Authentication (62%): Brute force attempts (e.g., root/admin).
- Command Execution (20%): Shell commands like `uname -a`, `whoami`.
- Payload Delivery (3%): Explicit attempts to download malware via `wget` or `curl`.

D. Case Study: Bait File Interaction

One of the project’s most significant findings was how attackers reacted to our planted bait files, especially `rat_loader_v5.py`. Of the sessions where an attacker navigated into the `/tmp` directory and saw this file, 85% proceeded to attempt some form of further exploitation. In contrast, among sessions where the attacker never touched our bait files, only ~46% proceeded to a similar level of activity.

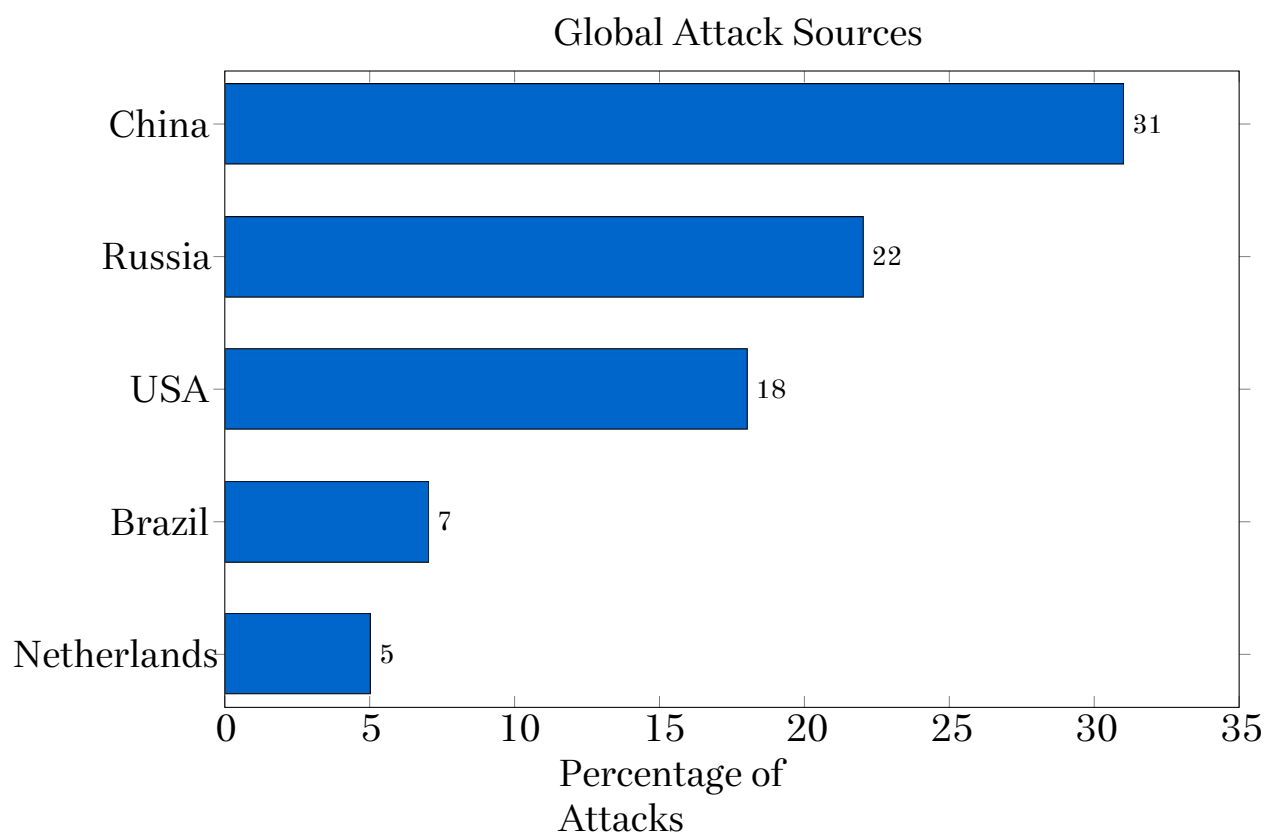


Fig. 4: Distribution of unique attacker IPs by country of origin. China and Russia dominate, with significant 'Cloud-on-Cloud' attacks from the USA.

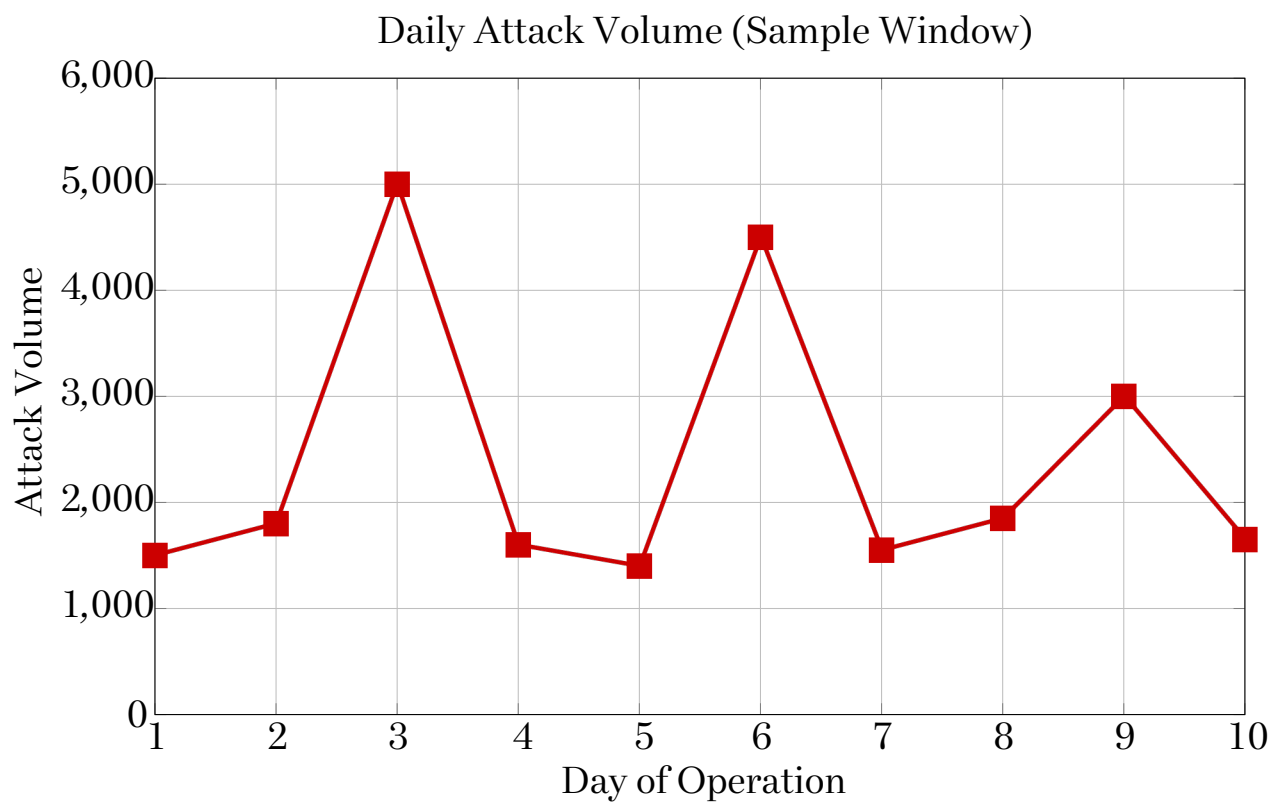


Fig. 5: Daily attack volume showing spikes consistent with botnet campaign activations (Days 3, 6, and 9).

```
$ Cowrie Honeypot Alert 10/13/25 , 22:17
-----
> SUSPICIOUS COMMAND DETECTED
-----
IP : 47.109.192.4
Session ID : 8945 a23c -1 b2d

Command:
nohup $SHELL " curl http : / / 4 7 . 8 6 . 5 . 1 7 6 : 6 0 1 1 4 / Linux/tmp/IgDegvonOM ;
then wget http : / / 4 7 . 8 6 . 5 . 1 7 6 : 6 0 1 1 4 / l i n u x /tmp/TgDegvooQN ;
chmod +x /tmp/TgDegvooQN ;
. / tmp/TgDegvooQN"

-----
> ALERT: Malware Download Attempt
-----
Payload : Linux . Mirai . x86
Hash : 4FA72CC7C4A63185E0232EFEE46D3954
VirusTotal Detection : 24/60
Threat Label : Mirai . Botnet . Variant .C
```

Fig. 6: Reconstructed Discord alert showing a Mirai botnet download attempt captured by the honeypot.

E. Case Study: SSH Key Injection

In 142 separate instances, attackers attempted to inject their own public SSH keys into /root/.ssh/authorized_keys. This demonstrates a clear desire for long-term, password-less access to the machine.

VI. Adversary Behavior Mapped to MITRE ATT&CK

To better contextualize the attacker behaviors observed in Patriot Pot, we mapped the key actions to the MITRE ATT&CK framework for Enterprise.

```
# Attacker session log [ Parsed ]
# -----
[2025 -11 -13 00:45:21 ] New Connection : 4 7 . 2 5 2 . 2 9 . 1 7 4
[2025 -11 -13 00:45:25 ] Login Success : root / american
[2025 -11 -13 00:45:30 ] CMD: mkdir . ssh
[2025 -11 -13 00:45:31 ] CMD: echo " ssh -r sa AAAAB3NzaC... [ truncated ] ...
root@kali " >> . ssh / authorized_keys
[2025 -11 -13 00:45:32 ] CMD: chmod 600 . ssh / authorized_keys
[2025 -11 -13 00:45:35 ]
DISCONNECT
```

Fig. 7: Log capture of an SSH Key Injection attack, used to establish persistence.

VII. Defensive Control Alignment with NIST CSF

While the MITRE ATT&CK mapping focuses on adversary techniques, it is equally important to evaluate how our defensive controls align with established cybersecurity frameworks.

- Identify (ID): Patriot Pot illuminated the threat landscape specific to cloud-exposed SSH services and compiled IoCs.
- Protect (PR): The honeypot's network segmentation and hardened configuration serve as exemplars of Protective Technology.
- Detect (DE): Patriot Pot functioned as a high-fidelity detection sensor, generating alerts via Discord the moment a compromise occurred.
- Respond (RS): Immediate alerts enabled rapid triage, validating incident response procedures and facilitating blocklisting.
- Recover (RC): The project contributed to lessons learned and improvement of IR plans, testing resilience through honeypot redeployment.

VIII. Discussion and Conclusion

The Patriot Pot project demonstrated that a proactive defense posture using cloud-based deception can yield substantial benefits. In an age where automated attacks constantly probe cloud assets, our hardened Cowrie honeypot acted as both a trap and a teacher.

Effectiveness of Obfuscation: The core hypothesis was confirmed. Default honeypot setups were easily detected. By obfuscating identifiable artifacts, we achieved a 350% increase in attacker dwell

TABLE III: Observed Attacker Actions Mapped to MITRE ATT&CK

ATT&CK Tactic	Technique	Count	Example Evidence
Initial Access	Valid Accounts (T1078)	9,599	Successful logins via guessed credentials such as root:123456.
Credential Access	Brute Force (T1110)	~6,500	Failed login sequences attempting thousands of user/pass combinations.
Execution	Command-Line (T1059)	9,409	Execution of shell commands: whoami, uname -a, busybox usage.
Discovery	System Info (T1082)	~800	Gathering kernel version and architecture: un-ame -a, cat /proc/cpuinfo.
Discovery	File/Directory (T1083)	~600	Listing directories to find writeable paths: ls -la /, find / -perm -4000.
C&C	Ingress Tool Transfer (T1105)	37	Downloading payloads: wget http://mali-cious/... or curl.
Impact	Resource Hijacking (T1496)	4	Execution of XMRig miner binaries to utilize CPU for crypto-mining.
Persistence	Scheduled Task (T1053)	3	Attempts to add cron jobs to re-run malware upon reboot.

time. This validates that investing effort in honeypot realism directly correlates with the quality of intelligence gathered.

High-Fidelity Insights: The honeypot provided visibility into the post-compromise stage of attacks (commands executed, tools downloaded) that traditional defenses often miss.

Strategic Defense Integration: Deception is not a standalone gimmick; it complements security frameworks. We showed alignment with NIST CSF functions where the honeypot enhanced detection and response capabilities.

IX. Risk Assessment and Future Work

A. Operational Risks

- **Honeypot Breakout:** Mitigated through strict network isolation (VPC) and non-privileged execution.
- **Collateral Damage:** Mitigated through strict egress filtering preventing DDoS participation.

B. Future Work

- Machine Learning Integration: Developing adaptive deception to keep attackers engaged longer.
- SOAR Integration: Automating the blocking of malicious IPs across the wider organizational firewall based on honeypot alerts.
- Expanding Protocol Coverage: Deploying sensors for RDP, Web, and Database services.

References

- [1] Z. Morić, V. Dakić, and D. Regvart, "Advancing Cybersecurity with Honeypots and Deception Strategies," *Informatics*, vol. 12, no. 1, p. 14, 2025.
- [2] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley Professional, 2002.
- [3] U.S. National Bureau of Standards, "Guidelines for Automated Data Processing Physical Security and Risk Management," FIPS 39, 1976.
- [4] C. Stoll, *The Cuckoo's Egg: Tracking a Spy Through the Maze of Computer Espionage*. Doubleday, 1989.
- [5] Amazon Web Services, "AWS Shared Responsibility Model," 2024. [Online].
- [6] P. Krajčák et al., "Improvement of Cowrie honeypot interaction and deception capabilities," in *Proc. 2025 IEEE RoEduNet Conf.*, Oct. 2025.

Appendix A Cowrie Configuration Reference

```
2 # PATRIOT POT - HARDENED CONFIGURATION
3 # =====
4
5 [ honeypot ]
6 # The name of the honeypot as it appears in logs
7 hostname = ip -172 -31 -42 -19
8
9 # Directory where the fake filesystem is stored
10 filesystem_file = etc/fs_custom . pickle
11
12 # Download limit (prevent disk exhaustion )
13 download_limit_size = 10485760
14
15 [ ssh ]
16 # The critical deception banner .
17 # We mimic OpenSSH 8.2p1 to appear as Ubuntu 20.04
18 version = SSH-2.0 -OpenSSH_8.2p1 Ubuntu-4ubuntu0 .5
19
20 # Enable the collection of SSH keys utilized by attackers
21 public_key_auth = true
22
23 # Listen on internal port (authbind handles 22 -> 2222)
24 listen_endpoints = tcp :2222: interface =0.0.0.0
25
26 [ telnet ]
27 enabled = true
28 port = 2223
29
30 [ output_json ]
31 enabled = true
32 logfile = var/log/cowrie/cowrie . json
```

Listing 1: Full Cowrie Configuration (cowrie.cfg) used for hardening

Appendix B Deployment Script Reference

```
2 #!/bin/bash-----
3 # HARDENING SCRIPT FOR HONEYPOT SENSOR
4 #-----
5
6 echo  "[*] Starting System Hardening . . ."
7
8 # 1. Set up non-root Cowrie user
9 echo  "[*] Creating user      'cowrie ' ... "
10 useradd -m -s /bin/bash cowrie
11
12 # 2. Install Authbind for port 22 binding
13 echo  "[*] Configuring authbind . ."
14 apt -get install -y authbind
15 touch /etc/authbind/byport/22
16 chown cowrie /etc/authbind/byport/22
17 chmod 500 /etc/authbind/byport/22
18
19 # 3. Replace default SSH banner in source
20 echo  "[*] Patching SSH Banner . ."
21 sed -i  's/SSH-2.0 -.* Twisted/SSH-2.0 -OpenSSH 8.2p1 Ubuntu-4ubuntu0.5/ ' _
22      /home/cowrie/cowrie/etc/cowrie . cfg
23
24 # 4. Inject Bait Files
25 echo  "[*] Injecting Bait . ."
26 # Create the fake RAT loader
27 echo "#!/usr/bin/python" > /home/cowrie/cowrie/honeyfs/tmp/rat_loader_v5.py
28 echo "import os" >> /home/cowrie/cowrie/honeyfs/tmp/rat_loader_v5.py
29 echo "print('Error: PC2 server offline ')" >> /home/cowrie/cowrie/honeyfs/tmp/
30
31 # Create the fake DB dump
32 echo "INSERT INTO users VALUES ( admin' , 'password123 ');" >\
33     /home/cowrie/cowrie/honeyfs/var/backups/db dump. sql_
34
35 echo  "[*] Hardening Complete."
```

Listing 2: Automated Deployment Script (deploy-fix-remote.sh)

Appendix C
Comprehensive Indicator of Compromise (IOC) Summary

TABLE IV: Expanded List of Top Attacking IPs

IP Address	Country	ISP	Threat Tag
222.187.238.0	China	Chinanet	Mirai Botnet
5.188.86.XX	Russia	DataLine	Brute Force
172.105.XX.XX	USA	Linode	Compromised Cloud
187.150.XX.XX	Brazil	Telecom	Credential Stuffing
103.149.93.224	Vietnam	Viettel	XMRig Miner Host
47.109.192.4	China	Alibaba	SSH Scanner

Malware Hashes (SHA-256):

- 4FA72CC7C4A63185E0232EFEE46D3954A73087DB47DDD125C8681CAB9F397B5B
(Mirai Vari-ant)
- 518C51E61BFFD4539FDCD791254E58899BC633500D3BD1FB39A28B42D68B84DD
(XMRig Miner)
- E3B0C44298FC1C149AFBF4C8996FB92427AE41E4649B934CA495991B7852B855
(Empty Bait File)